MODULE 9 – ASP.NET SECURITY

1. Introduction to ASP.NET Core Security

ASP.NET Core Security refers to the techniques and mechanisms used to protect web applications and APIs from unauthorized access, attacks and data breaches.

Security ensures that:

- Only authorized users can access protected resources.
- User identities are verified.
- Sensitive information is protected.
- Application data is handled securely.
- Malicious requests are prevented.
- Communication between client and server is protected.

Main Security Areas:

1. Authentication
2. Authorization
3. Password Security
4. JWT Authentication
5. Claims-Based Security
6. Role-Based Authorization
7. Policy-Based Authorization
8. HTTPS
9. CORS
10. CSRF Protection
11. XSS Prevention
12. SQL Injection Prevention
13. Data Protection
14. Secure Configuration

2. Authentication

Authentication is the process of verifying the identity of a user.

It answers the question:

Who are you?

Example:

A user enters:

- Email
- Password

The application checks whether the credentials are valid.

If they are valid, the application authenticates the user.

Common Authentication Methods:

1. Username and Password Authentication
2. Cookie Authentication
3. JWT Bearer Authentication
4. Multi-Factor Authentication (MFA)
5. External Authentication using Google or Microsoft

3. Authorization

Authorization is the process of determining what an authenticated user is allowed to access or perform.

It answers the question:

What are you allowed to do?

Example:

Consider a College Management System.

Admin:

- Add students.
- Delete students.
- Manage departments.
- View all records.

Student:

- View their profile.
- View their marks.
- Update permitted personal details.

The system checks the user's permissions before allowing an operation.

4. Authentication vs Authorization

Authentication:

- Verifies user identity.
- Determines who the user is.
- Usually happens before authorization.

Authorization:

- Checks access permissions.
- Determines what the user can do.
- Uses roles, claims, policies and requirements.

Example:

A user successfully logs in. This is authentication.

The application checks whether that user has permission to delete a student. This is authorization.

5. Security Middleware in ASP.NET Core

ASP.NET Core uses middleware to process HTTP requests and responses.

Authentication and authorization are implemented through middleware.

Important middleware:

UseAuthentication():

Identifies the current user using the configured authentication scheme.

UseAuthorization():

Checks whether the authenticated user has permission to access the requested resource.

Example:

```
None
app.UseAuthentication();

app.UseAuthorization();

app.MapControllers();
```

Important:
Authentication middleware must execute before authorization middleware.

6. Authentication Flow

1. User enters login credentials.
2. Client sends credentials to the login endpoint.
3. Server validates the credentials.
4. Server creates an authentication result, such as a JWT.
5. Server returns the token to the client.
6. Client sends the token with subsequent API requests.
7. Authentication middleware validates the token.
8. User identity and claims become available.
9. Authorization middleware checks access permissions.
10. The API returns the appropriate response.

7. Password Security

Passwords must never be stored as plain text in a database.

If a database is compromised, plain-text passwords can be exposed.

Instead, passwords should be stored using secure password hashing.

Hashing:

Hashing converts input data into a fixed-length output using a one-way algorithm.

Password hashing is designed to make recovering the original password computationally difficult.

Common password hashing algorithms:

- PBKDF2
- bcrypt
- Argon2
- scrypt

Hashing vs Encryption:

Hashing:

- One-way process.
- Does not normally allow recovery of the original value.
- Used for password verification.

Encryption:

- Reversible using the appropriate key.
- Used to protect data that must later be recovered.

ASP.NET Core Identity provides password hashing functionality.

Example:

```
None
using Microsoft.AspNetCore.Identity;

var hasher = new PasswordHasher<User>();

string hashedPassword = hasher.HashPassword(
    user,
    user.Password
);
```

Verify a password:

```
None
var result = hasher.VerifyHashedPassword(
    user,
    hashedPassword,
    enteredPassword
);

if (result == PasswordVerificationResult.Success)
{
    Console.WriteLine("Password is valid");
}
```

In real applications, prefer ASP.NET Core Identity's built-in password management rather than manually implementing password storage.

8. ASP.NET Core Identity

ASP.NET Core Identity is a membership system used to manage users, passwords, roles, claims and authentication-related information.

Features:

1. User registration.
2. User login.
3. Password hashing.
4. Password reset.
5. Email confirmation.
6. Role management.
7. Claims management.
8. Two-factor authentication.
9. Account lockout.
10. External login support.

Important classes:

- IdentityUser
- IdentityRole
- UserManager<TUser>
- RoleManager<TRole>
- SignInManager<TUser>
- IdentityDbContext

UserManager:

Manages users.

Examples:

```
None
await _userManager.CreateAsync(user, password);

var user = await _userManager.FindByEmailAsync(email);

await _userManager.AddToRoleAsync(user, "Admin");
```

SignInManager:

Manages sign-in operations.

Example:

```
None
var result = await _signInManager.PasswordSignInAsync(
    email,
    password,
    isPersistent: false,
    lockoutOnFailure: true
);
```

RoleManager:

Manages application roles.

Example:

```
None
await _roleManager.CreateAsync(new IdentityRole("Admin"));
```

9. JSON Web Token (JWT)

JWT stands for JSON Web Token.

It is a compact token format commonly used to transmit claims between parties.

JWT is widely used for authentication in Web APIs.

A JWT contains three parts:

```
None
Header.Payload.Signature
```

1. Header

Contains token type and signing algorithm information.

Example:

```
None
{
  "alg": "HS256",
```

```
  "typ": "JWT"
}
```

2. Payload

Contains claims about the user.

Example:

```
None
{
  "sub": "101",
  "email": "user@example.com",
  "role": "Admin"
}
```

3. Signature

Helps verify that the token has not been modified and was signed using the expected signing key.

10. JWT Authentication Flow

1. User submits login credentials.
2. Server validates the credentials.
3. Server creates a JWT containing required claims.
4. Server signs the token using a secret or private key.
5. Server sends the token to the client.
6. Client stores the token securely.
7. Client includes the token in future requests.
8. Server validates the token.
9. User identity is established.
10. Authorization rules determine access.

Example request:

```
None
GET /api/students HTTP/1.1
Authorization: Bearer eyJhbGciOi...
```

Important:
A JWT payload is encoded, not encrypted by default. Do not store passwords or sensitive secrets inside it.

11. JWT Configuration in ASP.NET Core

JWT authentication is commonly configured using the JWT Bearer authentication handler.

Install the required package when it is not already available:

```
None
dotnet add package Microsoft.AspNetCore.Authentication.JwtBearer
```

Configuration in Program.cs:

```
None
using Microsoft.AspNetCore.Authentication.JwtBearer;
using Microsoft.IdentityModel.Tokens;
using System.Text;

builder.Services.AddAuthentication(
    JwtBearerDefaults.AuthenticationScheme)
    .AddJwtBearer(options =>
    {
        options.TokenValidationParameters =
            new TokenValidationParameters
            {
                ValidateIssuer = true,
                ValidateAudience = true,
                ValidateLifetime = true,
                ValidateIssuerSigningKey = true,

                ValidIssuer = builder.Configuration["Jwt:Issuer"],

                ValidAudience =
builder.Configuration["Jwt:Audience"],

                IssuerSigningKey = new SymmetricSecurityKey(
                    Encoding.UTF8.GetBytes(
```

```
                    builder.Configuration["Jwt:Key"]!
                )
            )
        };
    });
```

Middleware:

```
None
app.UseAuthentication();

app.UseAuthorization();
```

JWT configuration values:

```
None
{
  "Jwt": {
    "Issuer": "MyApplication",
    "Audience": "MyApplicationClient",
    "Key": "UseAProperlyGeneratedSecretKey"
  }
}
```

For production, store signing keys in a secure secret store or other protected configuration source. Never commit real secrets to GitHub.

12. Creating a JWT Token

JWT tokens can be created using the JwtSecurityToken class.

Example:

```
None
var claims = new List<Claim>
{
```

```
    new Claim(ClaimTypes.NameIdentifier, user.Id.ToString()),
    new Claim(ClaimTypes.Email, user.Email),
    new Claim(ClaimTypes.Role, "Admin")
};

var key = new SymmetricSecurityKey(
    Encoding.UTF8.GetBytes(jwtKey)
);

var credentials = new SigningCredentials(
    key,
    SecurityAlgorithms.HmacSha256
);

var token = new JwtSecurityToken(
    issuer: issuer,
    audience: audience,
    claims: claims,
    expires: DateTime.UtcNow.AddMinutes(60),
    signingCredentials: credentials
);

string jwtToken = new JwtSecurityTokenHandler()
    .WriteToken(token);
```

Important classes:

- Claim: Represents information about a user.
- SymmetricSecurityKey: Represents a symmetric signing key.
- SigningCredentials: Specifies the signing key and algorithm.
- JwtSecurityToken: Represents the JWT.
- JwtSecurityTokenHandler: Creates and processes JWT strings.

13. JWT Token Validation

Token validation ensures that a token is trustworthy and valid.

Important validation properties:

- ValidateIssuer: Checks the token issuer.
- ValidateAudience: Checks the intended audience.
- ValidateLifetime: Checks token expiration.
- ValidateIssuerSigningKey: Checks the signature using the configured signing key.

If validation fails, the API rejects the token.

14. Claims-Based Authentication

A claim is a piece of information about a user.

Examples:

- User ID
- Email
- Name
- Role
- Department
- Permission

Example:

```
None
var claims = new List<Claim>
{
    new Claim(ClaimTypes.Name, "Divya"),
    new Claim(ClaimTypes.Email, "divya@example.com"),
    new Claim(ClaimTypes.Role, "Admin")
};
```

Claims can be used to identify users and make authorization decisions.

15. Role-Based Authorization

Role-based authorization restricts access according to a user's assigned role.

Common roles:

- Admin
- Manager
- Employee
- Customer
- Student

Example:

```
None
[Authorize(Roles = "Admin")]
[HttpDelete("{id}")]
public IActionResult DeleteStudent(int id)
{
    return NoContent();
}
```

Only authenticated users with the Admin role can access this endpoint.

Multiple roles:

```
None
[Authorize(Roles = "Admin,Manager")]
```

A user with either role can satisfy this role requirement.

16. Policy-Based Authorization

Policy-based authorization allows developers to define reusable authorization rules.

Register a policy:

```
None
builder.Services.AddAuthorization(options =>
{
    options.AddPolicy("AdminOnly", policy =>
        policy.RequireRole("Admin"));
});
```

Apply the policy:

```
None
[Authorize(Policy = "AdminOnly")]
[HttpGet("admin-data")]
public IActionResult GetAdminData()
```

```
{
    return Ok("Admin data");
}
```

Policies can check:

- Roles.
- Claims.
- Custom requirements.
- Custom authorization handlers.

17. Resource-Based Authorization

Resource-based authorization checks whether a user is permitted to perform an operation on a particular resource.

Example:

A customer should be able to cancel only their own booking.

Suppose:

- Customer A creates Booking 1.
- Customer B creates Booking 2.

Customer A should not be allowed to cancel Booking 2.

The application must verify resource ownership before allowing the operation.

Important components:

- IAuthorizationService
- IAuthorizationRequirement
- AuthorizationHandler<TRequirement>

Example:

```
None
public class BookingOwnerRequirement
    : IAuthorizationRequirement
{
}
```

A handler can compare the booking owner's user ID with the authenticated user's ID.

Resource-based authorization is useful when access depends on the actual database record rather than only a user's role.

18. [Authorize] Attribute

The Authorize attribute protects controllers and actions.

Example:

```
None
[Authorize]
[ApiController]
[Route("api/[controller]")]
public class StudentsController : ControllerBase
{
}
```

All actions in this controller require authentication unless an action explicitly allows anonymous access or another applicable configuration changes the behavior.

19. [AllowAnonymous] Attribute

AllowAnonymous permits an endpoint to be accessed without authentication.

Example:

```
None
[AllowAnonymous]
[HttpPost("login")]
public IActionResult Login()
{
    return Ok("Login endpoint");
}
```

Commonly used for:

- Registration.
- Login.
- Password reset requests.

- Public information endpoints.

20. 401 Unauthorized vs 403 Forbidden

401 Unauthorized:

The request does not contain valid authentication credentials.

Example:

- Missing JWT.
- Expired JWT.
- Invalid JWT.

403 Forbidden:

The user is authenticated but does not have the required permission.

Example:

- A Customer tries to access an Admin-only endpoint.

Remember:

401 = Authentication problem.

403 = Authorization problem.

21. Cross-Origin Resource Sharing (CORS)

CORS stands for Cross-Origin Resource Sharing.

It is a browser security mechanism that controls whether a frontend application can access resources from a different origin.

Example:

Frontend:

```
None
https://localhost:4200
```

Backend:

```
None
https://localhost:7057
```

These are different origins.

The browser may block the request unless the API allows the frontend origin through CORS configuration.

Configuration:

```
None
builder.Services.AddCors(options =>
{
    options.AddPolicy("AllowFrontend", policy =>
    {
        policy.WithOrigins("https://localhost:4200")
              .AllowAnyHeader()
              .AllowAnyMethod();
    });
});
```

Middleware:

```
None
app.UseRouting();

app.UseCors("AllowFrontend");

app.UseAuthentication();

app.UseAuthorization();

app.MapControllers();
```

Important:

- CORS is not authentication.
- CORS is not authorization.
- CORS is primarily enforced by browsers.

- Configure only trusted origins in production.
- Avoid allowing every origin when credentials are required.

22. Cross-Site Request Forgery (CSRF)

CSRF stands for Cross-Site Request Forgery.

It is an attack in which a malicious website tricks an authenticated user's browser into sending an unwanted request to another website.

Example:

1. User logs in to a banking website.
2. The browser stores an authentication cookie.
3. User visits a malicious website.
4. The malicious website causes the browser to send a request to the banking website.
5. The browser may automatically include the authentication cookie.
6. The banking website may process the unwanted request if proper protection is missing.

Prevention:

- Use anti-forgery tokens.
- Configure cookies with appropriate SameSite settings.
- Use Secure and HttpOnly cookie attributes.
- Validate request origin where appropriate.
- Avoid unsafe state-changing GET requests.

ASP.NET Core MVC supports anti-forgery validation.

Example:

```
None
[ValidateAntiForgeryToken]
[HttpPost]
public IActionResult TransferMoney()
{
    return Ok();
}
```

CSRF is especially important when authentication relies on automatically attached browser cookies.

23. Cross-Site Scripting (XSS)

XSS stands for Cross-Site Scripting.

It is an attack in which an attacker injects malicious scripts into web content that is viewed by other users.

Example:

An attacker submits malicious JavaScript as a comment.

If the application renders the content without proper output encoding, the script may execute in another user's browser.

Types of XSS:

1. Stored XSS: Malicious script is stored in the application.
2. Reflected XSS: Malicious input is reflected in a response.
3. DOM-Based XSS: Client-side JavaScript processes unsafe input and modifies the page in an unsafe way.

Prevention:

- Encode output.
- Avoid unsafe HTML rendering.
- Validate input.
- Use Content Security Policy (CSP).
- Avoid inserting untrusted input using unsafe DOM APIs.
- Use appropriate framework protections.

ASP.NET Core Razor automatically HTML-encodes ordinary variable output.

Example:

```
None
<p>@Model.UserComment</p>
```

Avoid using Html.Raw() with untrusted content.

24. SQL Injection

SQL Injection is an attack in which malicious input changes the intended structure of a SQL query.

Example of unsafe SQL:

```
None
string query =
    "SELECT * FROM Users WHERE Email = '" + email + "'";
```

An attacker may provide specially crafted input that changes the query's meaning.

Prevention:

- Use parameterized queries.
- Use Entity Framework Core safely.
- Avoid building SQL statements by concatenating untrusted input.
- Apply least-privilege database permissions.
- Validate input.

Safe EF Core example:

```
None
var user = await _context.Users
    .FirstOrDefaultAsync(u => u.Email == email);
```

Parameterized SQL example:

```
None
var users = await _context.Users
    .FromSqlInterpolated(
        $"SELECT * FROM Users WHERE Email = {email}")
    .ToListAsync();
```

EF Core parameterizes interpolated values in FromSqlInterpolated.

25. HTTPS and SSL/TLS

HTTPS stands for Hypertext Transfer Protocol Secure.

HTTPS uses TLS to protect communication between clients and servers.

It provides:

1. Confidentiality: Protects data from being read by unauthorized parties during transmission.
2. Integrity: Helps detect modification of data during transmission.

3. Authentication: Helps clients verify the server's identity using its certificate.

Example:

```
None
http://example.com
```

Unsecured HTTP.

```
None
https://example.com
```

HTTPS-protected communication.

In ASP.NET Core:

```
None
app.UseHttpsRedirection();
```

Production applications should use properly configured TLS certificates.

26. Data Protection in ASP.NET Core

ASP.NET Core Data Protection provides APIs for protecting sensitive application data.

It supports encryption and decryption through a protected data-protection mechanism.

Common uses:

- Protecting authentication-related data.
- Protecting cookies.
- Protecting temporary tokens.
- Protecting sensitive application payloads.

Example:

```
None
public class TokenService
{
    private readonly IDataProtector _protector;
```

```
    public TokenService(IDataProtectionProvider provider)
    {
        _protector = provider.CreateProtector("MyApplication");
    }

    public string Protect(string data)
    {
        return _protector.Protect(data);
    }

    public string Unprotect(string protectedData)
    {
        return _protector.Unprotect(protectedData);
    }
}
```

Required namespace:

```
None
using Microsoft.AspNetCore.DataProtection;
```

Data Protection is different from password hashing. It is designed for data that needs to be protected and later recovered.

27. Secure Cookies

Cookies are small pieces of data stored by a browser and sent with matching HTTP requests.

Cookies are commonly used for authentication and session-related functionality.

Important cookie security attributes:

HttpOnly:
Prevents ordinary client-side JavaScript from reading the cookie.

Secure:
Restricts the cookie to HTTPS connections.

SameSite:
Controls when cookies are sent with cross-site requests.

Example:

```
None
options.Cookie.HttpOnly = true;

options.Cookie.SecurePolicy =
    CookieSecurePolicy.Always;

options.Cookie.SameSite =
    SameSiteMode.Lax;
```

These settings should be selected according to the application's authentication and cross-site requirements.

28. Multi-Factor Authentication (MFA)

MFA stands for Multi-Factor Authentication.

It requires users to verify their identity using two or more different authentication factors.

Common factors:

1. Something you know: Password or PIN.
2. Something you have: Phone or security key.
3. Something you are: Fingerprint or face recognition.

Example:

1. User enters a password.
2. Application requests an additional verification code.
3. User enters the code.
4. Application completes authentication.

MFA improves security even if a password is compromised.

29. Account Lockout

Account lockout temporarily prevents further login attempts after a specified number of failed attempts.

Example:

If a user enters an incorrect password five times, the application may temporarily lock the account.

Benefits:

- Helps reduce brute-force attacks.
- Limits repeated password guessing.
- Protects user accounts.

ASP.NET Core Identity supports account lockout configuration.

30. Brute Force Attack

A brute-force attack attempts many possible passwords or credentials until the correct combination is found.

Prevention techniques:

- Rate limiting.
- Account lockout.
- Multi-factor authentication.
- Strong password requirements.
- Monitoring suspicious login attempts.
- Secure authentication design.

31. Rate Limiting

Rate limiting controls the number of requests accepted during a specified period.

It helps protect APIs from excessive requests and abuse.

Example:

```
None
builder.Services.AddRateLimiter(options =>
{
    options.AddFixedWindowLimiter("login", limiter =>
    {
        limiter.PermitLimit = 5;
        limiter.Window = TimeSpan.FromMinutes(1);
    });
});
```

Middleware:

```
None
app.UseRateLimiter();
```

Apply to an endpoint:

```
None
[EnableRateLimiting("login")]
[HttpPost("login")]
public IActionResult Login()
{
    return Ok();
}
```

For login endpoints, rate limiting should be combined with appropriate account protection and monitoring.

32. Input Validation and Sanitization

Input validation checks whether submitted data satisfies expected rules.

Examples:

- Email must have a valid format.
- Age must be within the allowed range.
- Name must not exceed the maximum length.
- Required fields must not be empty.

Example:

```
None
public class RegisterDto
{
    [Required]
    [EmailAddress]
    public string Email { get; set; } = string.Empty;

    [Required]
```

```
    [MinLength(8)]
    public string Password { get; set; } = string.Empty;

    [Range(18, 60)]
    public int Age { get; set; }
}
```

Validation does not replace output encoding, SQL parameterization or authorization. Each security problem requires the appropriate protection.

33. Secure Configuration

Applications often require sensitive configuration values.

Examples:

- Database passwords.
- JWT signing keys.
- API keys.
- External service credentials.

Do not store real production secrets in source-controlled appsettings.json files.

Common secure storage options:

- .NET User Secrets for local development.
- Environment variables.
- Azure Key Vault.
- Other approved secret-management systems.

Example configuration access:

```
None
var connectionString =
    builder.Configuration.GetConnectionString("DefaultConnection");
```

34. Logging and Security Monitoring

Security logging records important security-related events.

Examples:

- Successful login.
- Failed login.
- Unauthorized access attempts.
- Password reset requests.
- Account lockouts.
- Suspicious requests.

Example:

```
None
_logger.LogWarning(
    "Failed login attempt for {Email}",
    email
);
```

Never log:

- Passwords.
- Full authentication tokens.
- Secret keys.
- Sensitive personal information unnecessarily.

Security monitoring helps detect suspicious activity and investigate incidents.

35. Principle of Least Privilege

The Principle of Least Privilege means giving users and application components only the permissions they need to perform their responsibilities.

Example:

A Student should not have permission to delete all student records.

A database account used by a read-only reporting service should not have permission to modify database records.

Benefits:

- Reduces security risks.
- Limits damage caused by compromised accounts.
- Prevents unnecessary access.
- Improves application security.

36. Common Web Security Attacks

1. SQL Injection: Injecting malicious SQL through unsafe database queries.
2. XSS: Injecting malicious scripts into web pages.
3. CSRF: Tricking an authenticated browser into sending unwanted requests.
4. Brute Force: Repeatedly guessing passwords.
5. Session Hijacking: Stealing or misusing a user's session.
6. Phishing: Tricking users into revealing sensitive information.
7. Denial of Service (DoS): Making a service unavailable by overwhelming it.
8. Man-in-the-Middle (MITM): Intercepting or manipulating communication between parties.

37. Authentication vs Session-Based vs Token-Based Authentication

Session-Based Authentication:

- Server maintains session information.
- Browser commonly stores a session identifier in a cookie.
- Server uses that identifier to find the session.

Token-Based Authentication:

- Server issues a token after authentication.
- Client sends the token with subsequent requests.
- API validates the token to establish the user's identity.

JWT is a common token format.

Important:
Token-based authentication is not automatically more secure than session-based authentication. Both require secure storage, transport and validation.